Name: **Nilanjan Ghosh**

College: **SRM Institute of Science and Technology**

Course: **Java FSE Digital Nurture**

Week 1

Exercise 2: E-commerce Platform Search Function

Objective

To implement and analyze Linear Search and Binary Search algorithms for searching products in an e-commerce platform and compare their performance using asymptotic notation.

---

1. Understanding Asymptotic Notation

Big O Notation

Big O notation is a mathematical representation used to describe the performance or efficiency of an algorithm as the input size increases. It helps determine how the running time of an algorithm grows with the number of elements.

Big O notation is important because it:

- Measures algorithm efficiency.
- Helps compare different algorithms.
- Predicts performance for large datasets.
- Assists in selecting the most suitable algorithm.

Common Time Complexities

Complexity	Description
O(1)	Constant Time
O(log n)	Logarithmic Time
O(n)	Linear Time
$O(n^2)$	Quadratic Time

Search Operation Scenarios

Linear Search

Linear Search examines each element one by one until the target element is found.

Best Case: Target element is at the first position.
Time Complexity: O(1)

Average Case: Target element is somewhere in the middle.
Time Complexity: O(n)

Worst Case: Target element is at the last position or not present.
Time Complexity: O(n)

Binary Search

Binary Search works only on sorted data. It repeatedly divides the search space into two halves.

Best Case: Target element is found at the middle position.
Time Complexity: O(1)

Average Case: O(log n)

Worst Case: O(log n)

2. Setup

Product Class

A Product class is created with the following attributes:

- productId
- productName
- category

These attributes help identify and search products efficiently.

```
class Product {
   int productId;
   String productName;
   String category;

   Product(int productId, String productName, String category) {
      this.productId = productId;
      this.productName = productName;
```

```
        this.category = category;
    }
}
```

---

3. Implementation

Linear Search

Linear Search traverses the array from the beginning and compares each product's ID with the target ID.

Algorithm

1. Start from the first product.
2. Compare productId with targetId.
3. If matched, return the product.
4. Otherwise move to the next product.
5. Continue until the product is found or the array ends.

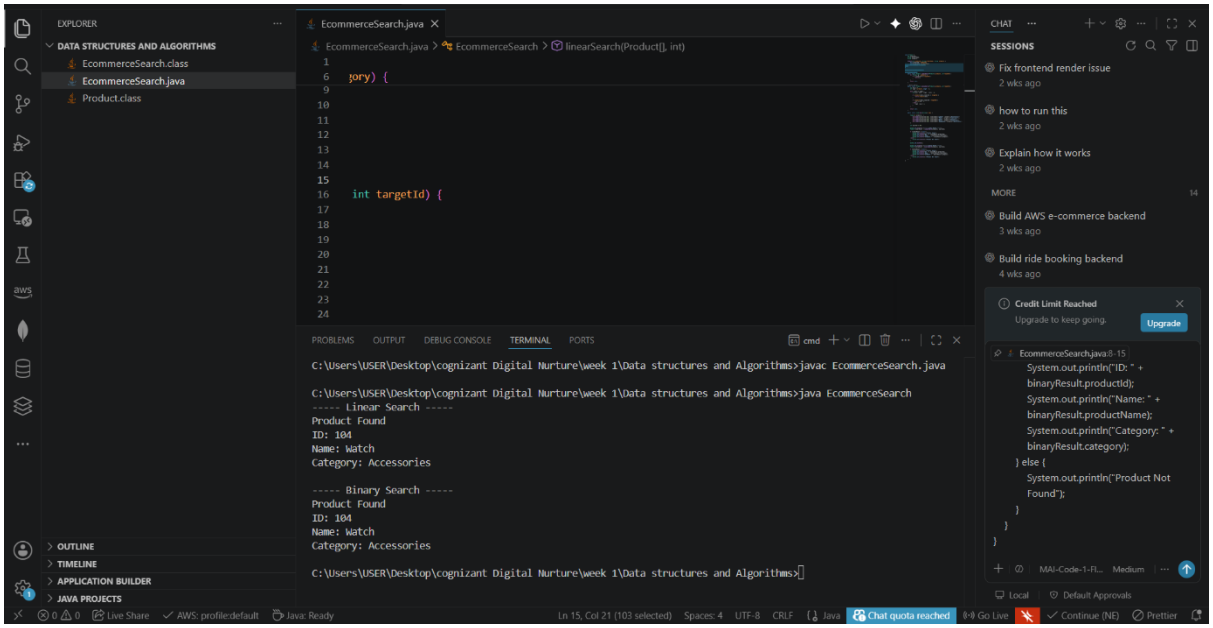
Binary Search

Binary Search is performed on a sorted array of products.

Algorithm

1. Set left and right pointers.
2. Find the middle element.
3. Compare middle productId with targetId.
4. If equal, return the product.
5. If targetId is greater, search the right half.
6. If targetId is smaller, search the left half.
7. Repeat until found or search space becomes empty.

OUTPUT:



4. Analysis

Time Complexity Comparison

Algorithm	Best Case	Average Case	Worst Case
Linear Search	O(1)	O(n)	O(n)
Binary Search	O(1)	O(log n)	O(log n)

Comparison

Linear Search

Advantages:

- Simple to implement.
- Works on unsorted data.

Disadvantages:

- Slow for large datasets.
- Requires checking many elements.

Binary Search

Advantages:

- Much faster for large datasets.
- Efficient searching due to repeated division of search space.

Disadvantages:

- Requires sorted data.

Suitable Algorithm for E-commerce Platform

Binary Search is more suitable for an e-commerce platform because product catalogs usually contain thousands or millions of products. Its time complexity of O(log n) allows products to be found much faster than Linear Search, which requires O(n) time in the average and worst cases.

Therefore, Binary Search provides better performance, scalability, and user experience for large product databases.

Conclusion

The Linear Search and Binary Search algorithms were studied and analyzed. Linear Search is simple but less efficient for large datasets. Binary Search offers significantly faster search performance with O(log n) complexity and is therefore the preferred choice for searching products in a large e-commerce platform.

Exercise 7: Financial Forecasting

Objective

To develop a financial forecasting tool using recursion to predict future values based on past growth rates and analyze the efficiency of the recursive algorithm.

---

1. Understanding Recursive Algorithms

Recursion

Recursion is a programming technique in which a method calls itself to solve a problem. A recursive solution typically consists of:

- **Base Case:** Stops the recursion.
- **Recursive Case:** Calls the same method with a smaller subproblem.

Recursion simplifies problems that can be broken into smaller, similar subproblems.

Advantages of Recursion

- Makes code shorter and easier to understand.
- Useful for mathematical computations.
- Naturally fits divide-and-conquer problems.

---

2. Setup

A method is created to calculate future financial value using:

- Current investment amount
- Annual growth rate
- Number of years

Formula:

Future Value = Present Value × (1 + Growth Rate)

This calculation is repeatedly applied using recursion.

---

3. Implementation

Java Program

```
class FinancialForecasting {

    public static double futureValue(double presentValue,
                                     double growthRate,
                                     int years) {

        // Base Case
        if (years == 0) {
            return presentValue;
        }

        // Recursive Case
        return futureValue(
                presentValue * (1 + growthRate),
                growthRate,
                years - 1);
    }

    public static void main(String[] args) {

        double presentValue = 10000;
        double growthRate = 0.10; // 10%
        int years = 5;

        double result = futureValue(
                presentValue,
                growthRate,
                years);

        System.out.println("Present Value: Rs." + presentValue);
        System.out.println("Growth Rate: " + (growthRate * 100) + "%");
        System.out.println("Years: " + years);
        System.out.println("Future Value: Rs." + result);
    }
}
```

Sample Output

```
Present Value: Rs.10000.0
Growth Rate: 10.0%
```

Years: 5
Future Value: Rs.16105.1

Working

For:

Present Value = 10000
Growth Rate = 10%
Years = 5

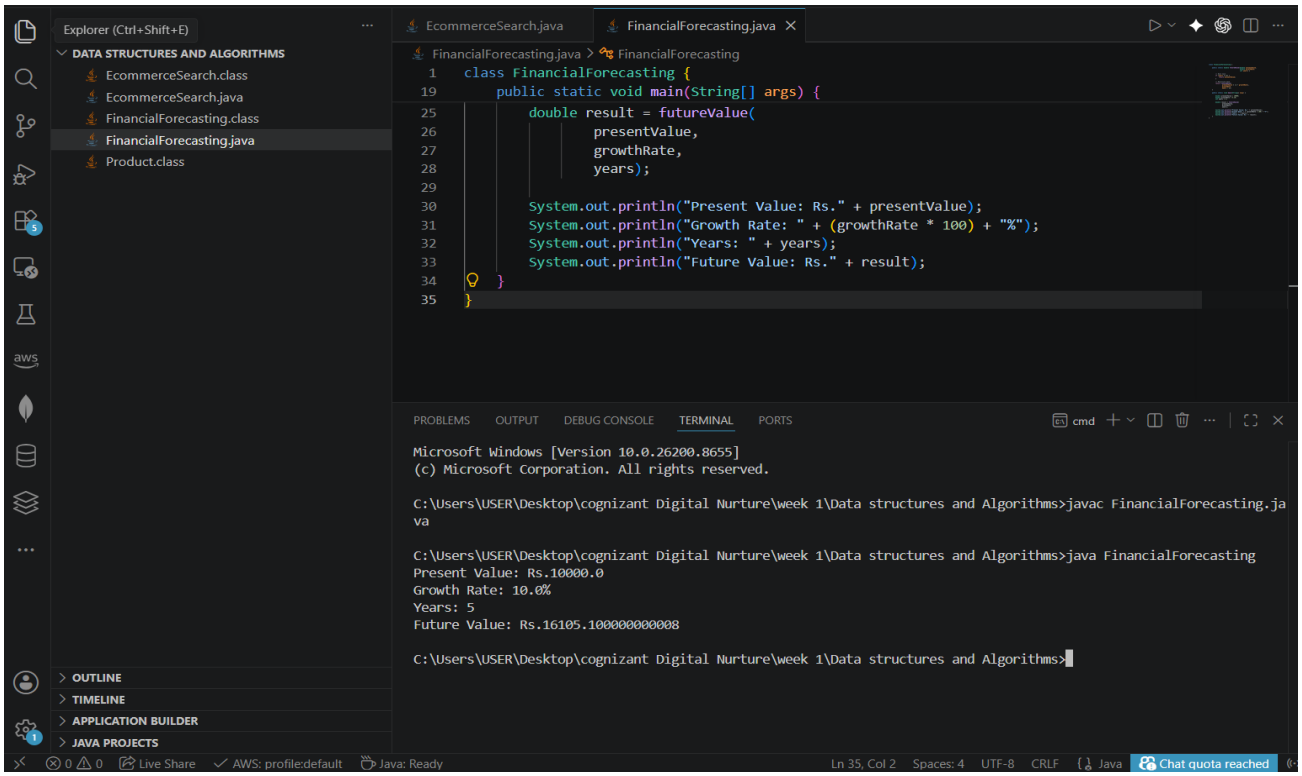
The recursive calls are:

10000 × 1.1 = 11000
11000 × 1.1 = 12100
12100 × 1.1 = 13310
13310 × 1.1 = 14641
14641 × 1.1 = 16105.1

Final Future Value:

Rs.16105.1

OUTPUT:



---

4. Analysis

Time Complexity

The recursive method makes one recursive call for each year.

Therefore:

T(n) = T(n-1) + O(1)

Time Complexity:

O(n)

where **n = number of years**.

Space Complexity

Each recursive call is stored in the call stack.

Space Complexity = O(n)

Optimization

The recursive solution can be optimized by:

1. Iterative Approach

Using a loop instead of recursion:

Time Complexity = O(n)
Space Complexity = O(1)

This avoids recursive stack overhead.

2. Mathematical Formula

Using compound interest directly:

Future Value = Present Value × (1 + Growth Rate)^Years

Java:

```
double futureValue =
      presentValue * Math.pow(1 + growthRate, years);
```

Complexity:

Time Complexity = O(1)
Space Complexity = O(1)

This is the most efficient approach for financial forecasting.

Conclusion

A recursive algorithm was implemented to forecast future financial values based on annual growth rates. The recursive solution successfully calculates future value with a time complexity of O(n). For large forecasting periods, iterative methods or the mathematical formula using Math.pow() provide better performance and reduce memory usage.